



Université Hassan II de Casablanca

Faculté des Sciences Ben M'Sick

Module : Optimisation

Pipeline Edge AI :

Fine-tuning, Benchmark d'Optimiseurs et Compression
d'un Modèle de Langage pour le Déploiement Embarqué

(MiniLM-L12 sur AG News)

Encadrante : Professeure Faouzia Benabbou

Réalisé par : Ayman Chabbaki, Mohammed Bendifi,

Imane Alouani, Marouane Daouki

Année universitaire 2025/2026

Remerciements

Nous tenons à exprimer notre sincère gratitude à notre encadrante, la Professeure Faouzia Benabbou, pour son accompagnement tout au long de ce projet. Ses conseils avisés et sa disponibilité ont été précieux pour mener à bien ce travail.

Nos remerciements s'adressent également à l'Université Hassan II de Casablanca et à la Faculté des Sciences Ben M'Sick pour la qualité de l'enseignement dispensé et les moyens mis à notre disposition. Enfin, nous saluons la contribution de chaque membre de notre équipe dont l'implication collective a permis de donner sa forme finale à ce rapport.

Table des matières

Remerciements	2
Introduction générale	4
Chapitre 1 : Contexte et problématique	5
1.1 L'optimiseur au coeur de la qualité d'entraînement	5
1.2 La nécessité de la compression pour le déploiement edge	5
1.3 Problématique	5
Chapitre 2 : Architecture du pipeline et préparation des données.....	6
2.1 Architecture générale du pipeline	6
2.2 Modèle et jeu de données.....	6
2.3 Prétraitement et tokenisation.....	6
Chapitre 3 : Benchmark comparatif des algorithmes d'optimisation.....	7
3.1 Méthodologie expérimentale	7
3.2 Résultats et analyses	7
3.3 Tableau récapitulatif.....	9
3.4 Analyse critique	10
Chapitre 4 : Compression du modèle par quantification	11
4.1 Principe de la quantification dynamique.....	11
4.2 Modèle retenu et procédure	11
Chapitre 5 : Évaluation en conditions Edge AI.....	12
5.1 Protocole d'évaluation.....	12
5.2 Résultats de l'évaluation edge	12
5.3 Discussion du compromis	13
Chapitre 6 : Discussion et analyse critique.....	14
6.1 Limites de l'étude	14
6.2 Perspectives d'amélioration.....	14
Conclusion générale et perspectives	15
Références bibliographiques	16

Introduction générale

L'essor récent des modèles de langage pré-entraînés a profondément transformé le paysage du traitement automatique des langues. Si les grandes architectures comme BERT ou GPT ont démontré des capacités remarquables sur une foule de tâches, leur déploiement sur des dispositifs à ressources limitées reste un défi majeur. C'est dans ce contexte que les Small Language Models (SLM), des modèles compacts obtenus par distillation ou conception allégée, gagnent en pertinence. Ils offrent un compromis attractif entre performance et légèreté, ouvrant la voie à des applications embarquées où la mémoire et la puissance de calcul sont des denrées rares.

Le déploiement en environnement edge computing, que ce soit sur des capteurs intelligents, des appareils mobiles ou des systèmes embarqués industriels, impose des contraintes drastiques. L'absence d'accélérateur GPU, la mémoire vive limitée et l'exigence d'une latence faible rendent impossible l'utilisation de modèles pleinement précisés en virgule flottante. Deux leviers principaux permettent de s'adapter à ces contraintes : le choix de l'algorithme d'optimisation lors de l'entraînement, qui influe sur la qualité finale et la vitesse de convergence, et la compression du modèle, notamment par quantification, qui réduit son empreinte mémoire et accélère l'inférence.

Ce rapport présente l'intégralité d'un pipeline Edge AI construit autour du modèle microsoft/MiniLM-L12-H384-uncased et du jeu de données AG News. Notre démarche s'articule en trois temps. Dans un premier temps, nous avons mis en place un benchmark comparatif de cinq optimiseurs (AdamW, Adafactor, Lion, LAMB et SGD) pour identifier le plus adapté au fine-tuning de ce modèle sur une tâche de classification de textes. Dans un deuxième temps, nous avons compressé le meilleur modèle obtenu par quantification dynamique en INT8. Enfin, nous avons évalué le compromis qualité-vitesse-mémoire de ce modèle compressé dans des conditions représentatives d'un déploiement edge sur CPU.

Chapitre 1 : Contexte et problématique

1.1 L'optimiseur au coeur de la qualité d'entraînement

Le choix de l'algorithme d'optimisation constitue l'une des décisions les plus impactantes lors du fine-tuning d'un modèle de langage. L'optimiseur détermine comment les gradients calculés lors de la rétropropagation sont exploités pour ajuster les poids du réseau. Chaque algorithme possède sa propre dynamique de convergence, sa sensibilité aux hyperparamètres, et sa consommation de ressources. AdamW, dérivé d'Adam avec un découplage plus rigoureux du déclin de poids, est devenu la référence dans l'écosystème des transformeurs. D'autres approches comme Adafactor, Lion, LAMB ou même SGD avec momentum présentent des caractéristiques différentes qui peuvent se révéler avantageuses selon le contexte.

La comparaison systématique de ces optimiseurs sur une même tâche et un même modèle permet d'éclairer leurs forces et faiblesses respectives. C'est précisément l'objectif du benchmark que nous détaillerons au chapitre 3.

1.2 La nécessité de la compression pour le déploiement edge

Un modèle de langage fine-tuné en précision FP32 occupe typiquement plus de 100 Mo en mémoire et nécessite des opérations matricielles coûteuses sur des tenseurs de grande dimension. Sur un dispositif embarqué dépourvu de GPU, ces caractéristiques se traduisent par des temps de réponse inacceptables et une consommation mémoire prohibitive. La quantification post-entraînement, et en particulier la quantification dynamique en INT8, offre une solution pragmatique. Elle convertit les poids du modèle en entiers 8 bits sans nécessiter de données de calibration, réduisant significativement l'empreinte mémoire tout en accélérant les calculs sur CPU grâce à des instructions optimisées.

1.3 Problématique

Ce projet se structure autour d'une question centrale : comment identifier le meilleur compromis entre performance prédictive, coût d'entraînement et faisabilité de déploiement sur dispositif contraint ? Plus concrètement, nous cherchons à déterminer quel optimiseur permet d'obtenir le modèle le plus performant en un temps raisonnable, puis à évaluer dans quelle mesure la quantification INT8 préserve cette performance tout en rendant le modèle exploitable dans un contexte edge.

Chapitre 2 : Architecture du pipeline et préparation des données

2.1 Architecture générale du pipeline

Le pipeline que nous avons développé s'organise en cinq blocs fonctionnels distincts. Le premier bloc assure le chargement et la préparation des données. Le deuxième bloc réalise le benchmark des cinq optimiseurs. Le troisième bloc applique la compression par quantification dynamique INT8 au meilleur modèle. Le quatrième bloc évalue le modèle compressé dans des conditions edge réalistes. Le cinquième bloc génère l'ensemble des visualisations. Cette architecture modulaire garantit la reproductibilité des expériences et permet d'isoler chaque étape pour un débogage facilité.

2.2 Modèle et jeu de données

Le modèle retenu est microsoft/MiniLM-L12-H384-uncased, un transformeur compact issu d'une distillation auto-supervisée de BERT-base. Avec seulement 12 couches et une dimension cachée de 384, il compte environ 33 millions de paramètres, ce qui en fait un candidat idéal pour le déploiement embarqué tout en conservant des capacités de représentation satisfaisantes.

Le jeu de données AG News est un corpus de classification de textes en quatre catégories : World, Sports, Business et Sci/Tech. Il contient initialement environ 120 000 exemples d'entraînement et 7 600 exemples de test. Afin d'accélérer les itérations expérimentales, nous avons effectué un sous-échantillonnage aléatoire (graine fixée à 42) à hauteur de 10 000 exemples d'entraînement et 2 000 exemples de test. Cette réduction permet de mener l'intégralité du benchmark en un temps raisonnable tout en conservant une distribution équilibrée entre les classes.

2.3 Prétraitement et tokenisation

La tokenisation est réalisée avec l'AutoTokenizer associé au modèle MiniLM. Chaque texte est tronqué ou complété (padding) à une longueur fixe de 128 tokens. Ce choix constitue un compromis classique pour des articles de presse courts : il capture l'essentiel du contenu sémantique sans surcharger inutilement les calculs. Les jeux de données tokenisés sont convertis au format tenseur PyTorch, prêts à alimenter la boucle d'entraînement.

Chapitre 3 : Benchmark comparatif des algorithmes d'optimisation

3.1 Méthodologie expérimentale

Nous avons fine-tuné le modèle MiniLM-L12 depuis ses poids pré-entraînés avec cinq optimiseurs différents : AdamW, Adafactor, Lion, LAMB et SGD. Chaque entraînement s'est déroulé sur trois époques avec une taille de batch de 32. Le taux d'apprentissage de base était fixé à $2e-5$, conforme aux recommandations usuelles pour le fine-tuning de transformeurs. Pour Lion, nous avons appliqué le facteur multiplicatif de 0,1 recommandé par ses auteurs, portant son taux effectif à $2e-6$.

Les métriques suivies couvrent plusieurs dimensions : la qualité prédictive (exactitude et F1 macro), la vitesse d'exécution (temps d'entraînement en secondes), la consommation de ressources (pic de mémoire CPU et GPU), et la stabilité de l'entraînement (écart-type de la perte calculé sur la seconde moitié des pas de gradient). Le meilleur modèle a été sélectionné sur la base du F1 macro maximal en validation.

3.2 Résultats et analyses

La figure 1 présente l'évolution de la perte d'entraînement au cours des trois époques pour chaque optimiseur. On y distingue clairement deux groupes. AdamW, Adafactor et Lion exhibent une courbe de décroissance régulière, témoignant d'un apprentissage effectif. En revanche, LAMB et SGD affichent une perte quasiment stationnaire, proche de 1,386 (soit $\ln(4)$), ce qui correspond à la perte théorique d'un modèle prédisant au hasard parmi quatre classes.

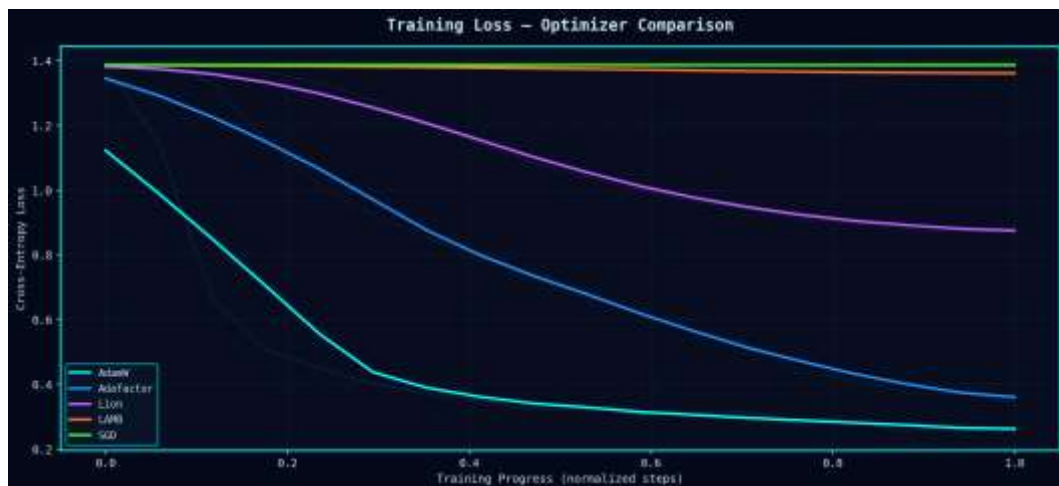


Figure 1 : Courbes de perte d'entraînement pour les cinq optimiseurs

La figure 2 montre la perte de validation par époque. AdamW atteint rapidement une valeur basse et stable dès la première époque (0,391), puis affine légèrement pour finir à 0,326. Adafactor démarre avec une perte plus élevée (0,853) mais rattrape son retard pour

converger vers 0,351. Lion progresse de manière plus linéaire, terminant à 0,856. Pour LAMB et SGD, la perte de validation reste bloquée autour de 1,38, confirmant l'absence d'apprentissage.



Figure 2 : Perte de validation par époque

Les courbes d'exactitude et de F1 macro, présentées à la figure 3, confirment cette dichotomie. AdamW atteint 0,9005 d'exactitude et 0,9013 de F1 macro. Adafactor se rapproche avec 0,8960 et 0,8969 respectivement. Lion parvient à un niveau honorable (0,8755 / 0,8766) malgré une convergence plus lente. LAMB culmine à 0,4695 d'exactitude et SGD reste bloqué à 0,2485, ce qui correspond exactement au niveau d'un classifieur aléatoire sur quatre classes équilibrées.

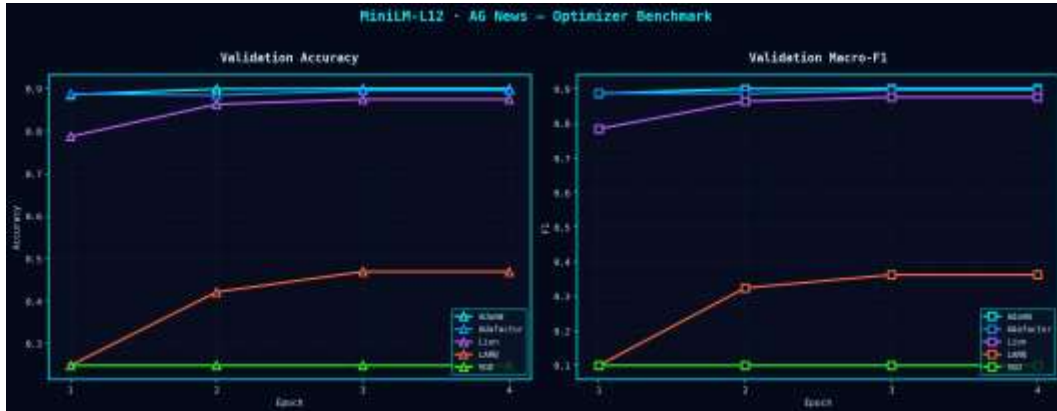


Figure 3 : Exactitude et F1 macro de validation par époque

La figure 4 offre une vue multidimensionnelle normalisée sous forme de radar. AdamW domine clairement sur les axes de qualité prédictive (exactitude et F1) tout en affichant une vitesse d'entraînement compétitive. Lion se distingue par une bonne stabilité de perte mais patit de sa convergence plus lente. SGD et LAMB apparaissent repliés au centre du radar, reflétant leurs résultats décevants sur tous les critères de qualité.

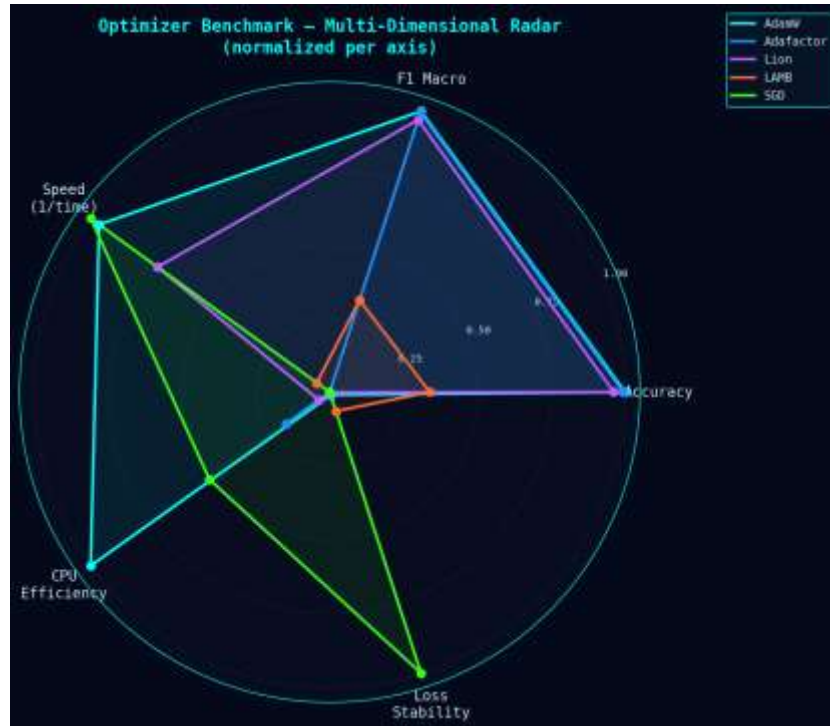


Figure 4 : Comparaison multidimensionnelle normalisée des optimiseurs

La figure 5 compare les temps d'entraînement et les pics de mémoire CPU. AdamW se distingue par un temps particulièrement court (62,27 s) pour une qualité maximale. Adafactor, malgré des performances proches d'AdamW, requiert environ deux fois plus de temps (125,94 s), ce qui constitue son principal handicap. Lion affiche un temps intermédiaire (71,40 s). SGD est rapide (61,13 s) mais n'apprend rien. LAMB est à la fois lent (119,12 s) et inefficace. La consommation mémoire est comparable pour tous les optimiseurs, variant entre 1 721 et 1 863 Mo de RAM CPU.

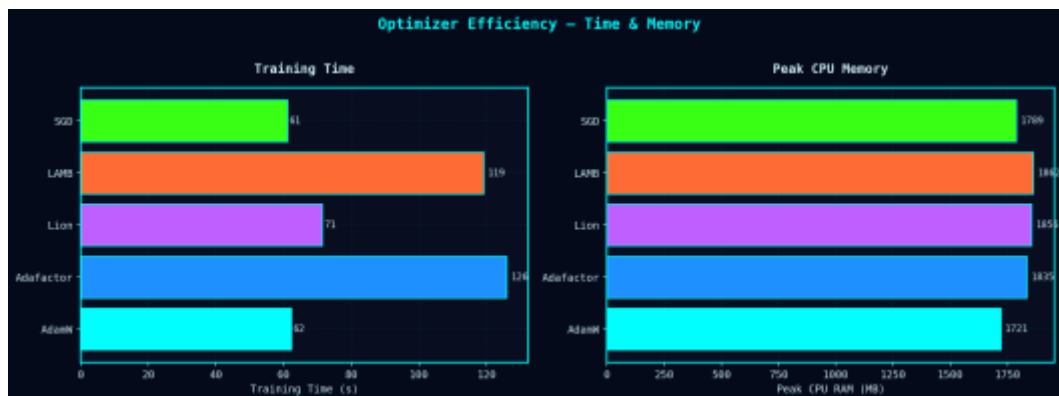


Figure 5 : Temps d'entraînement et pic de mémoire CPU

3.3 Tableau récapitulatif

Le tableau ci-dessous synthétise l'ensemble des résultats obtenus pour les cinq optimiseurs testés.

Optimiseur	Exactitude	F1 macro	Temps (s)	RAM CPU (Mo)	RAM GPU (Mo)	Écart-type
------------	------------	----------	-----------	--------------	--------------	------------

Optimiseur	Exactitude	F1 macro	Temps (s)	RAM CPU (Mo)	RAM GPU (Mo)	Écart-type
AdamW	0,9005	0,9013	62,27	1721,43	1272,39	0,0258
Adafactor	0,8960	0,8969	125,94	1835,07	1405,24	0,1117
Lion	0,8755	0,8766	71,40	1855,52	1662,35	0,0576
LAMB	0,4695	0,3620	119,12	1862,50	1926,98	0,0048
SGD	0,2485	0,0995	61,13	1788,79	1928,98	0,0003

Tableau 1 : Résultats finaux du benchmark des optimiseurs

3.4 Analyse critique

AdamW émerge comme l'optimiseur le plus performant sur cette tâche, avec le meilleur F1 macro (0,9013) et l'un des temps d'entraînement les plus courts. Cette supériorité s'explique par la capacité d'Adam à adapter individuellement les taux d'apprentissage de chaque paramètre, ce qui s'avère particulièrement efficace sur les architectures transformeur où les gradients varient fortement d'une couche à l'autre.

Adafactor approche les performances d'AdamW mais au prix d'un temps d'entraînement environ deux fois plus élevé. Sa gestion adaptative du taux d'apprentissage, bien qu'élégante en théorie, induit visiblement un surcoût computationnel dans notre configuration. Lion converge plus lentement au départ (sa perte reste élevée pendant la première époque) mais finit par rattraper un score correct. Son mécanisme de mise à jour par signe, plus brutal que celui d'AdamW, semble nécessiter davantage d'itérations pour pleinement s'exprimer.

L'échec de LAMB et SGD mérite une attention particulière. SGD reste bloqué à une exactitude de 0,2485, ce qui correspond exactement au niveau d'un classement aléatoire sur quatre classes. LAMB ne dépasse pas 0,47 d'exactitude. Il serait tentant d'attribuer ces résultats à un dysfonctionnement du pipeline, mais il s'agit en réalité d'un résultat attendu dans la littérature.

SGD et LAMB nécessitent généralement des taux d'apprentissage et des stratégies d'échauffement (warmup) très différents de ceux d'AdamW pour le fine-tuning de transformeurs. Le taux de $2e-5$, parfaitement calibré pour AdamW, est en réalité beaucoup trop faible pour permettre à SGD de sortir d'un optimum plat en seulement trois époques. LAMB, conçu pour l'optimisation en grand batch, souffre également de ce réglage inadapté. Signalons également que l'écart-type de perte très faible observé pour SGD (0,0003) et LAMB (0,0048) ne traduit pas une bonne stabilité d'entraînement mais au contraire une absence totale de progression : la perte reste bloquée près de $\ln(4)$, soit environ 1,386, qui est la perte théorique d'un modèle prédisant au hasard.

Chapitre 4 : Compression du modèle par quantification

4.1 Principe de la quantification dynamique

La technique de compression retenue est la quantification dynamique post-entraînement via la fonction `torch.quantization.quantize_dynamic` de PyTorch. Cette méthode convertit statiquement les poids des couches `nn.Linear` (qui concentrent l'essentiel des calculs d'un transformeur) en entiers signés sur 8 bits (INT8), tout en quantifiant dynamiquement les activations au moment de l'inférence.

Le principal avantage de cette approche réside dans son absence de nécessité de données de calibration. Contrairement à la quantification statique qui requiert un jeu de données représentatif pour déterminer les échelles de quantification, la méthode dynamique calcule ces échelles à la volée pour chaque batch d'entrée. Cela la rend particulièrement adaptée à un contexte académique où la simplicité de mise en oeuvre est un critère important.

4.2 Modèle retenu et procédure

Le modèle sélectionné pour la compression est celui ayant obtenu le meilleur F1 macro lors du benchmark, c'est-à-dire le modèle entraîné avec AdamW. Toutes les couches de type `nn.Linear` du modèle ont été quantifiées, couvrant ainsi les projections d'attention et les réseaux feed-forward qui constituent l'essentiel du coût computationnel. Les poids quantifiés ont été sauvegardés sur disque avec la configuration et le tokenizer, permettant une reconstruction complète du modèle compressé.

Chapitre 5 : Évaluation en conditions Edge AI

5.1 Protocole d'évaluation

L'évaluation finale compare le modèle FP32 (non compressé) et le modèle INT8 (quantifié) sur quatre familles de métriques : la taille sur disque, la taille en mémoire, la latence d'inférence et la qualité prédictive. La latence a été mesurée uniquement sur CPU, dans des conditions réalistes d'un déploiement edge sans accélérateur GPU. Le protocole comprend 20 itérations d'échauffement suivies de 200 itérations chronométrées, avec une taille de batch de 1 pour simuler une inférence requête par requête.

Il convient de préciser un point important pour la rigueur de ce rapport : l'exactitude de référence du modèle FP32 mesurée dans cette étape (0,884) est légèrement différente de celle obtenue à la fin du benchmark du chapitre 3 (0,9005). Cette disparité s'explique par une différence de protocole d'évaluation : l'évaluation finale a été réalisée sur un sous-échantillon de test distinct de 500 exemples tirés séparément, plutôt que sur l'ensemble de validation complet de 2 000 exemples utilisé pendant l'entraînement. Il ne s'agit donc pas d'une incohérence mais bien d'une différence méthodologique, que nous mentionnons pour la transparence scientifique.

5.2 Résultats de l'évaluation edge

La figure 6 présente le tableau de bord comparatif sous forme de quatre panneaux. Le premier panneau montre la réduction de taille sur disque : 127,29 Mo pour le modèle FP32 contre 66,90 Mo pour le modèle INT8, soit un gain de 1,90 fois. Le deuxième panneau illustre la latence moyenne d'inférence : 26,60 ms pour FP32 contre 16,80 ms pour INT8, ce qui correspond à une accélération de 1,58 fois. Le troisième et quatrième panneaux montrent la qualité prédictive : une exactitude de 0,884 contre 0,874 et un F1 macro de 0,8849 contre 0,8748.

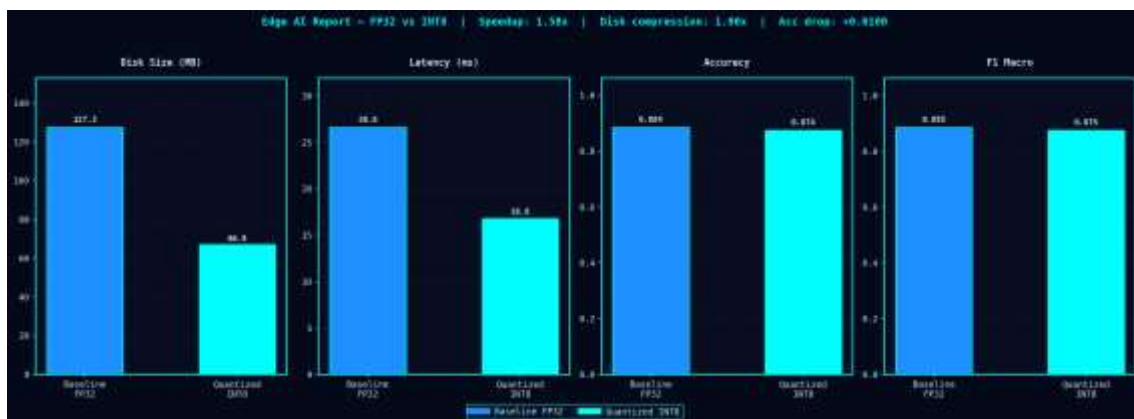


Figure 6 : Tableau de bord de compression Edge AI - FP32 vs INT8

Le tableau suivant synthétise l'ensemble des résultats obtenus.

Métrique	FP32 (référence)	INT8 (quantifié)
Taille sur disque (Mo)	127,29	66,90
Taille en mémoire (Mo)	127,27	45,54
Latence moyenne (ms)	26,60	16,80
Latence écart-type (ms)	6,85	1,74
Latence P95 (ms)	36,35	20,21
Exactitude	0,884	0,874
F1 macro	0,8849	0,8748

Tableau 2 : Comparaison FP32 vs INT8 (évaluation Edge AI)

5.3 Discussion du compromis

Les résultats mettent en lumière un compromis particulièrement favorable. La quantification dynamique en INT8 permet une réduction de taille sur disque d'environ 1,9 fois et une réduction de l'empreinte mémoire d'environ 2,79 fois (de 127,27 Mo à 45,54 Mo). La latence d'inférence s'améliore de 1,58 fois, ce qui est significatif pour une application temps réel. Ces gains s'accompagnent d'une perte de qualité très limitée : environ 1 point d'exactitude et 1 point de F1 macro.

Ce compromis nous semble tout à fait acceptable pour un déploiement sur dispositif contraint. Une exactitude de 0,874 sur AG News reste tout à fait honorable pour un modèle de moins de 67 Mo fonctionnant sur CPU sans accélérateur. Mentionnons également que la variabilité de la latence (écart-type et percentile 95) est notablement réduite pour le modèle INT8, ce qui constitue un avantage opérationnel pour les systèmes interactifs.

Chapitre 6 : Discussion et analyse critique

6.1 Limites de l'étude

Plusieurs limites méritent d'être soulignées pour contextualiser nos résultats. La première concerne le sous-échantillonnage des données. Avec seulement 10 000 exemples d'entraînement sur les 120 000 disponibles, nous favorisons probablement les optimiseurs qui convergent rapidement et pénalisons ceux qui auraient besoin de davantage de données ou d'itérations pour exprimer leur potentiel. Lion, par exemple, pourrait réduire l'écart avec AdamW sur un jeu complet et un nombre d'époques plus élevé.

La seconde limite réside dans le nombre fixe d'époques. Trois époques pour tous les optimiseurs n'est pas nécessairement équitable. AdamW converge vite et a pleinement profité de ces trois passes, tandis que Lion aurait probablement gagné à en effectuer davantage. Un protocole avec arrêt précoce basé sur la perte de validation aurait été plus juste.

La troisième limite, peut-être la plus significative, est qu'un seul jeu d'hyperparamètres a été testé par optimiseur. Le taux d'apprentissage de $2e-5$ est optimal pour AdamW mais clairement inadapté à SGD et LAMB. Un réglage spécifique de ces optimiseurs, avec des taux d'apprentissage plus élevés et des stratégies de warmup plus agressives, aurait probablement donné de résultats très différents. Notre benchmark compare donc plutôt des configurations que des algorithmes dans l'absolu.

6.2 Perspectives d'amélioration

Plusieurs pistes d'amélioration se dessinent. Sur le plan des optimiseurs, une recherche d'hyperparamètres par optimiseur (grid search ou méthode bayésienne) permettrait une comparaison plus équitable. Sur le plan de la compression, la quantification statique avec calibration pourrait offrir des ratios de compression supérieurs et une qualité préservée, au prix d'une mise en oeuvre légèrement plus complexe. Des techniques plus avancées comme la pruning (élagage structural) combinée à la quantification pourraient également être explorées.

Enfin, l'évaluation sur du matériel embarqué réel (Raspberry Pi, NVIDIA Jetson Nano, ou microcontrôleur avec accélérateur NP) apporterait une validation plus probante que notre simulation CPU. Les résultats obtenus sur un processeur x86 ne se transposent pas nécessairement à l'identique sur des architectures ARM, où les instructions de quantification peuvent avoir des performances différentes.

Conclusion générale et perspectives

Ce projet nous a permis de construire et d'évaluer un pipeline complet allant du fine-tuning d'un modèle de langage à son déploiement optimisé pour des conditions edge. Plusieurs enseignements se dégagent clairement de notre travail.

Tout d'abord, AdamW confirme son statut d'optimiseur de référence pour le fine-tuning de transformeurs compacts. Il offre la meilleure qualité prédictive tout en étant parmi les plus rapides, avec une robustesse face aux réglages par défaut qui en fait un choix sûr et efficace. Adafactor constitue une alternative viable lorsque la mémoire est une contrainte serrée, mais son temps d'entraînement plus long en limite l'attrait dans un contexte d'itérations rapides.

Ensuite, la quantification dynamique en INT8 s'avère une technique remarquablement efficace pour rendre un modèle de langage déployable sur dispositif contraint. Avec une perte de qualité d'environ 1 point d'exactitude pour un gain de vitesse de 1,58 fois et une réduction mémoire de 2,79 fois, le compromis est nettement favorable. L'absence de nécessité de données de calibration renforce encore l'intérêt pratique de cette méthode.

Enfin, notre benchmark souligne que certains optimiseurs réputés performants en théorie, comme LAMB et SGD, nécessitent un réglage spécifique pour être compétitifs dans le contexte précis du fine-tuning de transformeurs. Un taux d'apprentissage inadapté ou un manque de warmup suffisant les rend non fonctionnels, ce qui constitue un rappel important : la performance d'un algorithme d'optimisation est toujours relative à la qualité de son paramétrage.

Les perspectives ouvertes par ce travail sont nombreuses. L'extension du benchmark à d'autres optimiseurs récents (Shampoo, Sophie, ou les variantes adaptatives), l'exploration de techniques de compression plus agressives, et le déploiement effectif sur matériel embarqué constitueront les prolongements naturels de cette étude.

Références bibliographiques

- [1] Wang, W., Bao, H., Huang, S., Dong, L., & Wei, F. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. arXiv preprint arXiv:2002.10957.
- [2] Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. Advances in Neural Information Processing Systems, 28. Jeu de données AG News disponible sur Hugging Face Hub.
- [3] Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Liu, Y., ... & Le, Q. V. (2023). Symbolic Discovery of Optimization Algorithms. arXiv preprint arXiv:2302.06675.
- [4] You, Y., Li, J., Reddi, S., Hseu, J., Kumar, S., Bhojanapalli, S., ... & Hsieh, C. J. (2019). Large Batch Optimization for Deep Learning: Training BERT in 76 minutes. arXiv preprint arXiv:1904.00962.
- [5] Shazeer, N., & Stern, M. (2018). Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. arXiv preprint arXiv:1904.00962.
- [6] Hugging Face. (2024). Transformers Documentation. <https://huggingface.co/docs/transformers>
- [7] PyTorch. (2024). Quantization Documentation. <https://pytorch.org/docs/stable/quantization.html>